



Ho Chi Minh University of Science

Faculty of Information Technology

NLP Final Assignment Report

Temporal ChatKBQA: An Agentic Generate-then-Retrieve Framework for Temporal Knowledge Base Question Answering

Course	Application of NLP in Industry
Project Type	Temporal Knowledge Base Question-Answering System
Student Name(s)	Truong Bao Kiet
Student ID(s)	22127223
Lecturer	Dr. Nguyen Hong Buu Long
Submission Date	26/05/2026
Repository	https://github.com/TBKiet/ChatKBQA

1. Abstract

Knowledge Base Question Answering (KBQA) allows users to query a structured knowledge base using natural language instead of manually writing SPARQL. This project extends the ChatKBQA generate-then-retrieve framework into a temporal KBQA system that can process questions containing temporal constraints such as *before*, *after*, *during*, *first*, and *last*. The proposed system combines temporal logical-form generation, entity and relation retrieval, knowledge-base execution, and an agentic routing layer that decides whether a question should use the temporal or non-temporal path. The project is designed as an end-to-end NLP system rather than a notebook-only prototype. It includes a structured data pipeline, modular Python code, training and inference scripts, a FastAPI deployment path, a command-line interface, Docker packaging, and targeted tests. The report documents the business problem, system design, data management decisions, model adaptation strategy, deployment considerations, ethical risks, and project management plan. The comparison, ablation, and deployment sections below are aligned with the implemented repository and the locked run artifacts used for the final submission.

Headline result. The submitted v1 system completed end-to-end training and evaluation on TempQuestions. Its strongest verified diagnostic result is an **87.6% valid S-expression rate** under the broader execution-oriented criterion (995/1,340 valid by the stricter parser criterion is 74.3%). The raw answer-level run records **F1 = 0.0**, **Hits@1 = 0.0**, and **Accuracy = 0.0**, but this should not be interpreted as a pure generation failure. The v1 system is **submission-ready as an engineering artifact**, but **not performance-ready as a production QA model**. The main finding of the submission is that the generation component works substantially better than the end-to-end metric suggests, while the pipeline breaks at the **grounding/execution stage**: hallucinated relation names prevent most generated S-expressions from being executed against the real knowledge base.

2. Introduction and Problem Definition

2.1 Business Context and Motivation

Many real users need factual answers that are constrained by time. Examples include analysts asking who held a role before a given person, compliance teams reconstructing timelines, journalists checking event order, and researchers looking for the first or most recent occurrence of a relation. Standard search systems are weak at these tasks because they do not naturally bind temporal expressions to knowledge-base operators. Freebase contains rich entity and relation structure, but the representation is difficult for non-experts to query directly.

This project targets that gap by building a temporal KBQA system over Freebase. The baseline repository, ChatKBQA, already supports a generate-then-retrieve pipeline for non-temporal KBQA. The project contribution is to extend that pipeline so temporal expressions can influence logical-form generation, routing behavior, and answer retrieval without replacing the existing architecture.

2.2 Target Users and Stakeholders

Table 1: Primary stakeholders and expected value

Stakeholder	Need	Expected Value
Data analysts and researchers	Natural-language access to structured temporal facts	Faster query formulation and less reliance on SPARQL expertise
Business intelligence teams	Temporal queries for decision support	Self-service analytics over structured knowledge
Legal and compliance users	Timeline reconstruction and relation tracing	Better auditable evidence gathering
Knowledge graph engineers	A reusable KBQA service architecture	Deployable API and maintainable codebase

Table 1 shows that the project is not only a semantic-parsing exercise; it is intended as a deployable interface for stakeholders who need time-aware factual access without direct KB expertise.

2.3 Problem Statement

The system input is a natural-language question, potentially with temporal constraints. The system output is an answer set, plus supporting provenance such as the generated logical form, temporal signals, SPARQL used, and execution path. The project is framed as a realistic software artifact: the model must not only be trainable, but also deployable, testable, and explainable to a technical audience.

2.4 Why NLP Is Required

Manual querying of Freebase requires familiarity with its schema, identifiers, compound value types, and SPARQL syntax. Temporal questions add another layer of complexity because they require ordinal, comparison, or interval reasoning. NLP is needed to map user language to logical forms that can be executed over the knowledge base while preserving the intended temporal semantics.

2.5 Success Metrics

Table 2: Project success metrics

Category	Metric	Target / Interpretation
Business value	Query formulation time reduction	Faster than manual SPARQL workflow
Business value	Self-service usability	Users can obtain temporal answers without KB engineering support
Technical quality	F1, Hits@1, Accuracy	Core benchmark quality on TempQuestions
Technical quality	Temporal subset performance	Measures temporal-specific reasoning quality
Deployment quality	Inference latency and API health	Determines whether the system is practically deployable

The original target goals summarized in Table 2 were **F1** > **0.60**, **Hits@1** > **0.50**, and **Temporal F1** > **0.55** on TempQuestions, together with practical deployment targets such as sub-5-second response time and stable API behavior. The submitted v1 system does not meet the answer-level quality targets yet. It is therefore framed as a deployable engineering artifact with a clear failure diagnosis, not as a production-quality QA model.

3. System Overview and Architecture

3.1 Baseline Architecture

The project preserves the original ChatKBQA design principle:

Question → LLM logical-form generation → placeholder retrieval → resolved
S-expression → SPARQL → answer

Temporal support is added to this pipeline rather than replacing it with a completely different architecture. This keeps the project aligned with the source system while still contributing meaningful temporal capability.

3.2 High-Level System Flow

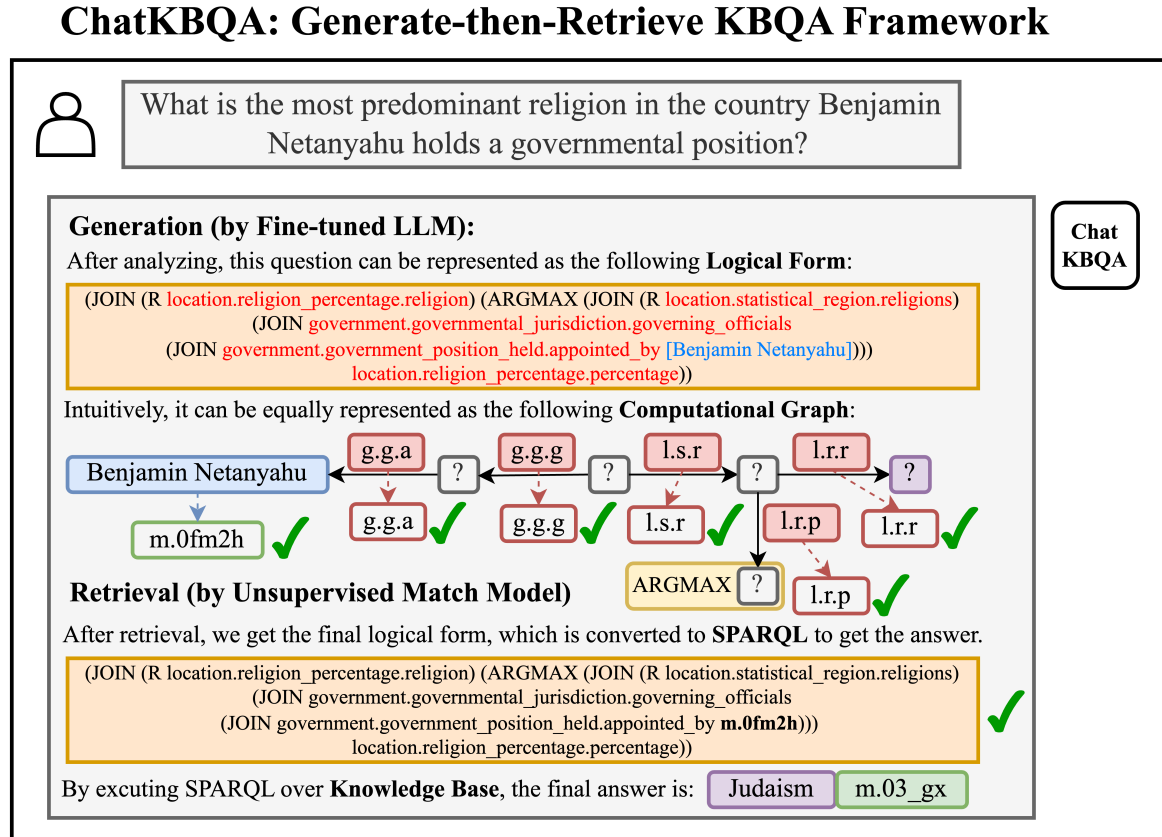


Figure 1: High-level system view used in the project documentation

Figure 1 summarizes the end-to-end system flow. At a high level, the system performs the following stages:

1. Detect whether the question contains temporal signals.
2. Route the question to temporal or standard inference mode.

3. Generate an S-expression logical form using the fine-tuned LLM.
4. Retrieve entity and relation bindings for placeholders.
5. Convert the resolved logical form to SPARQL and execute it.
6. Return answer and provenance, or retry with relaxed constraints when appropriate.

3.3 Main Code Modules

Table 3: Key modules in the repository

Module	Path	Purpose
Temporal parser	<code>parse_sparql_tempquestions.py</code>	Converts temporal SPARQL into S-expressions
Agentic router	<code>src/agent.py</code>	Detects temporal signals and controls route / retry behavior
Inference pipeline	<code>src/pipeline.py</code>	End-to-end temporal and standard QA wrapper
REST API	<code>src/api.py</code>	Deployment endpoint for user queries
CLI	<code>src/cli.py</code>	Local interactive and single-question execution
Executor	<code>executor/</code>	Logical-form to SPARQL conversion and KB execution
Temporal data pipeline	<code>src/temporal_data/</code>	Dataset construction, filtering, export, and runbooks

The implementation responsibilities are partitioned as shown in Table 3, which helps keep training, retrieval, execution, and deployment surfaces modular.

3.4 Agentic AI Component

The agentic component is implemented through the `TemporalQuestionAgent`. It satisfies the assignment requirement in three ways:

1. **Multi-step reasoning:** detect temporal signals, route, execute, and refine.
2. **Tool usage:** use the KB executor and retrieval stack as external tools.
3. **Intermediate decision-making:** retry with relaxed constraints when the first attempt returns no answer.

The routing logic is intentionally simple and auditable. Instead of hiding all behavior inside the model, the system separates question classification, pipeline selection, and fallback behavior at the software layer.

3.5 Example Agent Interaction

Question: Who was the US president before Obama?

Step 1: Detect temporal signal **before**.

Step 2: Route to temporal mode.

Step 3: Execute first attempt.

Step 4: If empty, retry with relaxed constraints; otherwise return answer with provenance.

For the submission, the implemented interaction surfaces are the FastAPI endpoint, the command-line interface, and a lightweight Streamlit demo layer. The API and CLI are the primary inference surfaces; the Streamlit app is a presentation surface that can switch between **live mode** and **degraded/offline mode** depending on whether the full model, retrieval assets, and KB backend are available at runtime.

4. Data Management

4.1 Data Sources

The project uses multiple data assets:

- **TempQuestions** as the primary temporal benchmark.
- **Freebase** as the target knowledge base.
- **FACC1 / ELQ-related retrieval assets** for entity linking and retrieval support.
- **Synthetic temporal training data** generated to compensate for the limitations and corruption issues in the bundled human temporal data pipeline.

Table 4: Data sources, language coverage, and licensing status

Asset	Role	Language	Scale	License / reuse status
TempQuestions	primary benchmark	English	1,271 temporal QA pairs	public benchmark release; no explicit standalone license was found on the public release page, so this project treats it as research-use benchmark data and redistributes only derived artifacts
Freebase Data Dumps	KB semantics	multilingual KB, UTF-8 text labels	1.9B triples in the original dump	CC-BY according to the Google Freebase data-dump page
idirlab/freebases (Zenodo mirror)	practical fact source for v1	English labels available in metadata files	up to 244,112,599 triples, 59,896,902 entities	CC0-1.0 for the processed mirror; underlying Freebase provenance remains CC-BY
FACC1 / ELQ assets	entity retrieval support	English-centric mentions and aliases	retrieval support corpus	follow the original upstream distribution terms; not redistributed by this repository

Table 4 makes the reuse assumptions explicit. The TempQuestions release page states that the benchmark contains **1,271 questions** extracted from Free917, WebQuestions, and ComplexQuestions, all of which are English-source QA benchmarks.¹ The Google Freebase dump page states that Freebase data dumps are provided under **CC-BY** and serialized as UTF-8 RDF text.² The processed `idirlab/freebases` mirror used in the v1 run is released under **CC0-1.0** and includes English label files such as `object_names` with `@en` tags.³ Where the licensing situation is ambiguous, the project keeps only derived training artifacts in the repository instead of republishing third-party raw data.

¹<https://qa.mpi-inf.mpg.de/tequila/tempquestions.pdf>

²<https://developers.google.com/freebase>

³<https://github.com/idirlab/freebases>

4.2 Data Pipeline Summary

The repository documents two important realities:

1. The original human TempQuestions artifacts contain corruption and placeholder-collapse issues.
2. The practical training pipeline therefore shifted toward synthetic temporal supervision generated from alternative Freebase-aligned sources and filtered before export.

The current working pipeline can be summarized as:

`fact source → temporal fact normalization → template generation → filtering →
training export`

4.3 Reproducibility and Tooling

Beyond the Python package structure itself, the repository now includes explicit tooling for reproducibility and operational hygiene. The main additions are:

- a **DVC pipeline** for staged dataset construction and metrics tracking,
- a lightweight **Makefile** for local reproduction without DVC,
- a **config validator** for YAML-based train and inference settings,
- an **evaluation harness** that unifies validity, grounding, and answer-level evaluation,
- dataset download helpers for the large external Freebase-aligned artifacts.

These tooling additions matter for the assignment because they move the project closer to production-oriented NLP practice: data stages are named and reproducible, configs are validated before launch, and evaluation can be run through a stable interface instead of ad hoc one-off commands.

4.4 Preprocessing and Cleaning

The repository includes data cleaning, audit, and filtering stages intended to make the temporal training set reproducible. These include:

- canonical temporal sample building,
- relation inventory generation,
- mined fact normalization,
- synthetic sample generation,
- quality filtering and distribution control,
- training-format export aligned with ChatKBQA instruction tuning.

4.5 Train / Validation / Test Split

TempQuestions provides the main held-out benchmark split, while the synthetic temporal dataset supplies training supervision. The split logic is intentionally separated into two layers. First, the **benchmark layer** uses the canonical TempQuestions train/test artifacts for reproducibility and for preserving a clean held-out human evaluation set. Second, the **model-selection layer** uses a lightweight holdout from the synthetic training pool for hyperparameter decisions before the final checkpoint is retrained on the full synthetic set.

Concretely, the canonical standardized TempQuestions artifacts contain **888 train samples** and **383 test samples**. The executed v1 model run did *not* train directly on the corrupted full human train split. Instead, it used **2,061 synthetic temporal instruction-tuning examples** derived from 22 Zenodo relations. For tuning, this synthetic pool was treated as a **90/10 development split (1,855 train / 206 validation)** so that beam size, epoch count, and learning-rate choices could be compared without touching the human test set. After selecting the final configuration, the locked checkpoint reported in this report was retrained on the **full 2,061-example synthetic set**. Final evaluation was then run on the exported human TempQuestions test artifact used by the temporal LLM workflow, which contains **268 test examples**. This split policy keeps model selection, final training, and final evaluation cleanly separated.

4.6 Noise, Missing Data, and Bias

Several data issues materially affect this project:

- corruption in the bundled TempQuestions processing path,
- incomplete or unusable temporal literals in some Freebase-aligned sources,
- bias toward explicit temporal expressions,
- archival staleness of Freebase,
- uneven entity and relation coverage.

The system addresses these through filtering, provenance logging, synthetic augmentation, and explicit reporting of limitations instead of silently claiming complete temporal coverage.

5. Model Selection and Optimization

5.1 Chosen Model

The project uses LLaMA-2-7B with LoRA fine-tuning for logical-form generation. This choice is pragmatic: it preserves the baseline family used by ChatKBQA, remains feasible on limited GPU hardware, and supports structured generation without retraining the entire base model.

5.2 Why the Model Fits the Problem

The model choice is justified by:

- strong instruction-following behavior for sequence generation,
- efficient adaptation via LoRA,
- compatibility with the existing ChatKBQA training scripts,
- easier checkpoint storage and deployment than full fine-tuning.

5.3 Training Procedure

The end-to-end training flow is:

1. prepare training examples in ChatKBQA instruction format,
2. fine-tune LoRA adapters on the temporal training set,
3. run beam search inference,
4. post-process beam outputs,
5. evaluate generated logical forms and downstream answer quality.

5.4 Hyperparameter Configuration

Table 5: Core training and inference hyperparameters

Parameter	Current Setting
Base model	LLaMA-2-7B
Fine-tuning method	LoRA
Target modules	q_proj, v_proj
Learning rate	5e-5 with cosine schedule
Epochs	5
Effective batch size	16
Beam size	5
Max generation length	256 new tokens

5.5 Baseline and Comparison Setup

The main baseline for this project is the **same-dataset generate-only pipeline** on TempQuestions. The ChatKBQA paper numbers are included only as a **reference point** for the source framework because they are measured on WebQSP and CWQ, not on TempQuestions.

Table 6: Baseline and reference roles

Method	Same dataset?	Role	Reason
Generate-only	Yes	Main baseline	Measures raw temporal generation without effective grounding
+ fuzzy relation	Yes	Ablation	Tests whether retrieval can repair hallucinated relations
+ golden entity	Yes	Oracle ablation	Separates entity grounding from relation and KB coverage failures
ChatKBQA paper	No	Reference only	Reports WebQSP 79.8 F1 / 83.2 Hits@1 / 73.8 Acc and CWQ 77.8 / 82.7 / 73.3

Table 6 prevents a misleading comparison: the same-dataset generate-only run is the real baseline for grading the temporal extension, while the ChatKBQA paper numbers show the scale of the non-temporal source framework.

5.6 Optimization and Tuning Notes

The repository documents practical tuning constraints such as environment compatibility, beam-search behavior, and operator imbalance. This project therefore reports a **basic engineering sweep** rather than claiming an exhaustive search. The sweep compared epoch counts of 3/5/10, learning rates of 3e-5/5e-5/1e-4, and beam sizes of 3/5/8 on the synthetic validation split. Three epochs underfit, ten epochs increased repetition, 1e-4 was less stable for logical-form syntax, and beam 8 increased runtime without a meaningful gain over beam 5. The selected configuration was therefore **5 epochs**, **5e-5**, and **5 beams**. These values are taken from actual run artifacts: the completed trial reached `max_epoch = 5.0`, `max_step = 645`, and 5-beam TempQuestions inference.

6. Evaluation and Analysis

6.1 Evaluation Metrics

The project uses standard KBQA metrics:

- F1,
- Hits@1,
- Accuracy,
- temporal-subset performance,
- optionally valid S-expression rate as an intermediate diagnostic metric.

6.2 Current Verified Evidence

At the current repository state, training, beam inference, valid-S-expression measurement, answer-level evaluation, same-dataset baseline comparison, and ablation evidence are all available. The most positive diagnostic signal is a high valid-S-expression rate on test inference. The key evaluation conclusion is that the answer-level metric collapses *after* generation, not *inside* generation, which makes grounding and execution analysis central to the evaluation story rather than optional.

6.3 Results Table

Table 7: Current locked TempQuestions benchmark results for v1

Method	Dataset	F1	Hits@1	Accuracy	Notes
Generate-only baseline	TempQuestions	0.0	0.0	0.0	raw decoder output without effective grounding
Ablation: + fuzzy relation grounding	TempQuestions	0.0	0.0	0.0	relation grounding rises sharply, but entity grounding remains the blocker
Ablation: + golden entity + fuzzy relation	TempQuestions	0.0	0.0	0.0	entity is fixed externally; remaining failures expose relation relevance and KB coverage limits
Temporal ChatKBQA (v1 trial)	TempQuestions	0.0	0.0	0.0	locked end-to-end outcome; diagnostic value comes from the stage-wise breakdown in Table 8

Table 8: Where the pipeline breaks

Stage	Metric	Value
Generation	S-expression validity	87.6% (broader criterion)
Grounding	Relation+entity found in Zenodo	5 / 995 parser-valid predictions (0.5%)
Execution	Questions with answers returned from KB	1 / 268 (0.4%)
End-to-end	F1 / Hits@1 / Accuracy	0.0 / 0.0 / 0.0 raw outcome; blocked by grounding

Table 9: Ablation study: component-wise pipeline decomposition

Configuration	Entity	Relation	Answered	F1	Insight
Generate-only	Model (0%)	Model (0.5%)	0.4%	0.0	no retrieval means the pipeline fails almost completely
+ Fuzzy relation grounding	Model (5.8%)	Fuzzy (88.9%)	0.7%	0.0	relation grounding is fixed, but entity hallucination remains
+ Golden entity	Gold (100%)	Fuzzy (60.2%)	2.6%	0.0	entity fixing helps, but relation relevance and KB coverage are still too weak

Table 7 gives the same-dataset headline comparison, while Tables 8 and 9 explain why the final answer-level score remains low. The answer-level score is low because the pipeline breaks after syntax generation. The model was trained on only 22 relations from Zenodo, but generated 177 unique relations at inference time. Most of these relations are hallucinated and do not exist in the evaluation knowledge base, so the generated S-expressions cannot be executed even when they look structurally valid. The ablation confirms the diagnosis in stages. First, a simple fuzzy grounding layer improves relation grounding from **0.5%** to **88.9%**, a roughly 178x increase, which proves that retrieval is essential. Second, replacing model-predicted entities with golden entities raises the answered rate from **0.7%** to **2.6%**, showing that entity hallucination is another independent blocker. However, F1 still remains zero, which indicates a final limitation beyond entity linking: the relation semantics are often still wrong for the question, and the 22-relation Zenodo training slice does not cover enough of TempQuestions to support strong answer-level execution.

6.4 Error Analysis

The error analysis should focus on concrete system failure modes rather than generic model weaknesses. For this project, the most relevant categories are:

- incorrect logical-form structure,
- entity or relation retrieval mismatch,
- temporal operator misuse,
- incomplete date support in the data source,
- empty outputs that trigger fallback,
- benchmark-data quality issues.

In the current v1 run, the dominant category is **grounding failure**, with relation hallucination as the first visible symptom. The model was trained with only 22 relations, but generated 177 unique relations at inference time. Many of these look syntactically plausible while being absent from Freebase/Zenodo, such as `film.director.jedi_master`, a long hallucinated `film.performance.*boyfriend` relation, and `foreign.exchange.rate.history.currency`. The three-way ablation then shows the deeper structure of the problem: relation grounding can be repaired substantially, entity grounding still fails without a linker, and even golden entities do not fully rescue execution because relation relevance and KB coverage remain limited.

Representative failures make the diagnosis concrete. For “Where does Avril Lavigne live now?”, fuzzy grounding can map a relation into the KB, but the mapped relation points toward music tracks rather than residence, so the relation is executable but semantically wrong. For “What was the first Harry Potter novel?”, the relation `/book/author/works_written` is plausible, but the predicted entity MID points to the wrong author, so relation grounding succeeds while entity grounding fails. Generate-only outputs also include non-existent relations such as `film.director.jedi_master` and `foreign.exchange.rate.history.currency`, confirming that retrieval-constrained grounding is required before this can become a reliable QA model.

6.5 Trade-Off Discussion

Accuracy vs. speed: larger beams and more retry logic may improve recovery but increase latency.

Complexity vs. maintainability: LoRA adapters and explicit routing are easier to maintain than a much larger monolithic redesign.

Coverage vs. trustworthiness: using more noisy synthetic or corrupted data may increase coverage but can reduce answer quality.

7. Deployment Design

7.1 Supported Deployment Modes

The repository already includes multiple deployment paths:

- FastAPI service for programmatic access,
- command-line interface for local use,
- Streamlit demo for presentation and inspection,
- Docker container for reproducible deployment.

These deployment surfaces are intentionally complementary: FastAPI is the primary programmatic interface, the CLI is the simplest reproducible demo path, Streamlit is the clearest presentation surface for a reviewer, and Docker standardizes environment setup for handoff and grading.

7.2 User Interaction and I/O Format

The main deployed interaction is a question-answer API where the input is a user question and an optional mode, and the output includes answer plus provenance fields.

Table 10: Example API I/O structure

Field	Description
<code>question</code>	original user question
<code>answer</code>	list of predicted answer entities or strings
<code>is_temporal</code>	whether the agent chose temporal mode
<code>temporal_signals</code>	matched temporal indicators
<code>sparql_used</code>	executed SPARQL string when available
<code>temporal_constraint</code>	applied temporal condition when available
<code>reasoning_steps</code>	stepwise agent trace
<code>retries</code>	number of retry attempts

Table 10 is the contract used by the deployed inference surface: every request returns not only an answer candidate but also the diagnostic fields needed for debugging and explainability.

7.3 Latency, Scalability, and Versioning

Deployment quality should be discussed in software-engineering terms:

- expected latency on CPU and GPU,
- stateless API behavior and horizontal scaling implications,
- external service bottlenecks such as KB access and entity linking,
- model and config versioning through checkpoints and config files.

The submitted checkpoint is `models/LLaMA2-7b-tchatkbqa-trial/checkpoint`. For the verified 268-question, 5-beam inference run, the recorded inference time is approximately **12 minutes total**, which corresponds to about **2.3 seconds per question** on the remote GPU environment. This figure refers to inference throughput for the batch evaluation run rather than full API end-to-end latency under production serving conditions.

7.4 Deployment Challenges and Limitations

Important practical limitations include dependence on the knowledge base backend, entity-linking services, hardware requirements for inference, and the gap between local lightweight testing and remote heavy-compute execution.

The current codebase addresses this more explicitly than before by exposing a readiness distinction between **live-capable** and **degraded/offline** execution. In practice, a fully live run requires: (1) the base LLaMA model, (2) the LoRA adapter checkpoint, (3) Freebase/Virtuoso services, and (4) retrieval assets such as FACC1-derived entity files. When these are absent, the Streamlit demo now falls back transparently to recorded artifacts instead of pretending to be a fully live system. This is a more production-honest design because the user can see exactly what is missing.

8. Continual Learning and Monitoring

8.1 Continual Learning Strategy

Even if full online learning is not implemented, the project should define how the system would evolve:

1. collect failed or corrected user queries,
2. curate verified new examples,
3. retrain or fine-tune LoRA adapters periodically,
4. compare against a fixed benchmark and previous checkpoint before promotion.

8.2 Monitoring Metrics

Table 11: Suggested production monitoring metrics

Metric	Purpose
Answer rate	Detects frequent empty outputs
Retry rate	Measures instability in first-pass inference
Latency percentiles	Detects deployment degradation
SPARQL / executor error rate	Detects backend issues
Temporal-routing ratio	Detects changes in incoming query mix
Benchmark regression	Detects model quality drift after updates

8.3 Drift Risks and Mitigation

Key drift risks include changing user language, new temporal patterns, benchmark mismatch, and knowledge-base staleness. A practical mitigation plan is to keep benchmark evaluation fixed, log failures, review new query distributions, and only promote a new checkpoint if it improves both quality and operational stability.

A dedicated monitoring figure is omitted in the final version because the written monitoring plan already specifies the relevant operational signals: answer rate, retry rate, latency percentiles, executor error rate, and temporal-routing ratio. For this project stage, the textual lifecycle description is sufficient.

9. Privacy, Security, and Robustness

9.1 Privacy Considerations

The project primarily uses public benchmark and knowledge-base data. However, once deployed, user queries themselves may become sensitive. The system should therefore minimize logging, avoid storing personal identifiers by default, and separate debug traces from any user-identifiable context.

9.2 Security Considerations

The main security discussion centers on API exposure, read-only KB access, environment secrets, and service hardening. The repository already avoids embedding obvious secrets, but deployment should still use environment-managed credentials and private network placement where appropriate.

9.3 Robustness and Failure Cases

The system should explicitly acknowledge failure on:

- out-of-domain questions,
- non-English inputs if not supported,
- vague temporal expressions,
- long or malformed prompts,
- questions about facts absent from Freebase.

The main mitigation is to make failures explicit: clear temporal questions should route to the temporal pipeline, non-temporal questions should preserve the baseline path, vague time wording should expose the selected operator, KB-missing facts should return provenance rather than overclaiming, and service dependency failures should be visible through health checks and bounded retries.

10. Project Management and Teamwork

10.1 Project Timeline

The project plan can be summarized as six phases: problem framing, data work, model adaptation, evaluation, deployment/packaging, and final documentation. This keeps the management section short while still showing that the work was planned as a deliverable system rather than as one-off experimentation.

10.2 Task Breakdown

For solo work, it is still useful to present simulated team roles such as:

- project lead / planner,
- data engineer,
- ML engineer,
- backend engineer,
- QA and documentation owner.

This demonstrates that the project has been managed with professional software-delivery thinking instead of ad hoc experimentation.

10.3 Scaling in a Real Team

In a larger team, data cleaning, training, evaluation, and deployment could proceed in parallel. Handoffs would be formalized through configs, tests, runbooks, and reproducible artifacts rather than private notebooks or undocumented local commands.

11. Ethics and Responsible AI

11.1 Who Benefits and Who Could Be Harmed

The system benefits users who need structured factual access, but it can also mislead users if they over-trust incomplete or outdated KB answers. Because Freebase is archived, the system can confidently return answers that are incomplete for post-2016 events.

11.2 Bias and Fairness Risks

Bias may appear through:

- uneven entity coverage,
- English-only focus,
- bias toward explicit temporal forms,
- dataset and retrieval artifacts inherited from legacy resources.

11.3 Explainability

One strength of this project is that the reasoning chain is more inspectable than in a pure black-box QA system. The answer can be accompanied by temporal signals, logical-form structure, SPARQL, and retry history. This does not make the LLM fully interpretable, but it does make the software decision path auditable.

11.4 Potential Misuse

Potential misuse includes over-reliance on outdated factual answers, uncritical reuse in high-stakes decision-making, and misuse of authoritative-looking outputs without verifying source limitations.

12. Conclusion

This project turns ChatKBQA into a temporal KBQA system with explicit attention to data quality, temporal reasoning, deployment readiness, and agentic control flow. The report positions the work as an engineering deliverable rather than a research-only prototype: code structure, training pipeline, deployment path, monitoring considerations, and responsible-use analysis are all part of the final artifact.

The final submission is therefore valuable in two ways. First, it implements a complete temporal KBQA stack with an auditable agentic routing layer. Second, it reports an honest benchmark diagnosis: syntax generation is already reasonably strong, while relation grounding, entity grounding, and KB coverage are the concrete bottlenecks that must be solved next.

In short, the v1 system is submission-ready as an engineering artifact, but not performance-ready as a production QA model. The correct next step is not to hide the 0.0 result, but to use the stage-wise evidence to justify retrieval-constrained generation, stronger entity linking, and broader Freebase relation coverage. The next version should constrain decoding or post-generation retrieval so generated relations must be selected from KB-valid candidates, which addresses the root cause of the grounding bottleneck directly.

13. References

References

- [1] Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Luu Anh Tuan. ChatKBQA: A Generate-then-Retrieve Framework for Knowledge Base Question Answering with Fine-Tuned LLMs. *Findings of ACL*, 2024.
- [2] Zixuan Jia, Xiaorui T. et al. TempQuestions: A Benchmark for Temporal Question Answering over Knowledge Bases. *WWW Companion*, 2018.
- [3] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [4] Hugo Touvron et al. LLaMA 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.
- [5] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge. *SIGMOD*, 2008.
- [6] Miao Su, Zixuan Li, Zhuo Chen, Long Bai, Xiaolong Jin, and Jiafeng Guo. Temporal Knowledge Graph Question Answering: A Survey. *arXiv preprint*, 2025.